

Chapter 9. Refactoring, testing, and debugging

This chapter covers

- Refactoring code to use lambda expressions
- Appreciating the impact of lambda expressions on object-oriented design patterns
- Testing lambda expressions
- Debugging code that uses lambda expressions and the Streams API

In the first eight chapters of this book, you saw the expressive power of lambdas and the Streams API. You were mainly creating new code that used these features. If you have to start a new Java project, you can use lambdas and streams immediately.

Unfortunately, you don't always get to start a new project from scratch. Most of the time you have to deal with an existing code base written in an older version of Java.

This chapter presents several recipes that show you how to refactor existing code to use lambda expressions to gain readability and flexibility. In addition, we discuss how several object-oriented design patterns (including strategy, template method, observer, chain of responsibility, and factory) can be made more concise thanks to lambda expressions. Finally, we explore how you can test and debug code that uses lambda expressions and the Streams API.

In [chapter 10](#), we explore a more wide-ranging way of refactoring code to make the application logic more readable: creating a domain-specific language.

9.1. Refactoring for improved readability and flexibility

From the start of this book, we've argued that lambda expressions let you write more concise and flexible code. The code is more concise because lambda expressions let you represent a piece of behavior in a more compact form compared with using anonymous classes. We also showed you

Other formats available



Audiobook



ces let you write even more concise
ass an existing method as an argument

Your code is more flexible because lambda expressions encourage the style of behavior parameterization that we introduced in [chapter 2](#). Your code can use and execute multiple behaviors passed as arguments to cope with requirement changes.

In this section, we bring everything together and show you simple steps for refactoring code to gain readability and flexibility, using the features you learned in previous chapters: lambdas, method references, and streams.

9.1.1. Improving code readability

What does it mean to improve the readability of code? Defining good readability can be subjective. The general view is that the term means “how easily this code can be understood by another human.” Improving code readability ensures that your code is understandable and maintainable by people other than you. You can take a few steps to make sure that your code is understandable to other people, such as making sure that your code is well documented and follows coding standards.

Using features introduced in Java 8 can also improve code readability compared with previous versions. You can reduce the verbosity of your code, making it easier to understand. Also, you can better show the intent of your code by using method references and the Streams API.

In this chapter, we describe three simple refactorings that use lambdas, method references, and streams, which you can apply to your code to improve its readability:

- Refactoring anonymous classes to lambda expressions
- Refactoring lambda expressions to method references
- Refactoring imperative-style data processing to streams

9.1.2. From anonymous classes to lambda expressions

The first simple refactoring you should consider is converting uses of anonymous classes implementing one single abstract method to lambda expressions. Why? We hope that in earlier chapters, we convinced you that anonymous classes are verbose and error-prone. By adopting lambda expressions, you produce code that’s more succinct and readable. As shown in [chapter 3](#), here’s an anonymous class for creating a `Runnable` object and its lambda-expression counterpart:

```
Runnable r1 = new Runnable() {                                1
    public void run(){
        System.out.println("Hello");
    }
}
```

Other formats available



- 1 Before, using an anonymous class
- 2 After, using a lambda expression

```
em.out.println("Hello");                                    2
```

But converting anonymous classes to lambda expressions can be a difficult process in certain situations.^[1] First, the meanings of `this` and `super` are different for anonymous classes and lambda expressions. Inside an anonymous class, `this` refers to the anonymous class itself, but inside a lambda, it refers to the enclosing class. Second, anonymous classes are allowed to shadow variables from the enclosing class. Lambda expressions can't (they'll cause a compile error), as shown in the following code:

¹

This excellent paper describes the process in more detail:

<http://dig.cs.illinois.edu/papers/lambdaRefactoring.pdf>.

```
int a = 10;
Runnable r1 = () -> {
    int a = 2;                                1
    System.out.println(a);
};
Runnable r2 = new Runnable(){
    public void run(){
        int a = 2;                            2
        System.out.println(a);
    }
};
```

- **1 Compile error**
- **2 Everything is fine!**

Finally, converting an anonymous class to a lambda expression can make the resulting code ambiguous in the context of overloading. Indeed, the type of anonymous class is explicit at instantiation, but the type of the lambda depends on its context. Here's an example of how this situation can be problematic. Suppose that you've declared a functional interface with the same signature as `Runnable`, here called `Task` (as might occur when you need more-meaningful interface names in your domain model):

```
interface Task {
    public void execute();
}
public static void doSomething(Runnable r){ r.run(); }
public static void doSomething(Task a){ r.execute(); }
```

Other formats available



class implementing `Task` without a

```
doSomething(new Task() {
    public void execute() {
        System.out.println("Danger danger!!");
    }
});
```

```
}
});
```

But converting this anonymous class to a lambda expression results in an ambiguous method call, because both `Runnable` and `Task` are valid target types:

```
doSomething(() -> System.out.println("Danger danger!!")); 1
```

- **1 Problem; both `doSomething(Runnable)` and `doSomething(Task)` match.**

You can solve the ambiguity by providing an explicit cast (`Task`):

```
doSomething((Task)() -> System.out.println("Danger danger!!"));
```

Don't be turned off by these issues, though; there's good news! Most integrated development environments (IDEs)—such as NetBeans, Eclipse, and IntelliJ—support this refactoring and automatically ensure that these gotchas don't arise.

9.1.3. From lambda expressions to method references

Lambda expressions are great for short code that needs to be passed around. But consider using method references whenever possible to improve code readability. A method name states the intent of your code more clearly. In [chapter 6](#), for example, we showed you the following code to group dishes by caloric levels:

```
Map<CaloricLevel, List<Dish>> dishesByCaloricLevel =
    menu.stream()
        .collect(
            groupingBy(dish -> {
                if (dish.getCalories() <= 400) return CaloricLevel.DIET;
                else if (dish.getCalories() <= 700) return CaloricLevel.NORMAL;
                else return CaloricLevel.FAT;
            }));
```

You can extract the lambda expression into a separate method and pass it as an argument to `groupingBy`. The code becomes more concise, and its

Other formats available

 Audiobook 

```
Map<CaloricLevel, List<Dish>> dishesByCaloricLevel =
    menu.stream().collect(groupingBy(Dish::getCaloricLevel)); 1
```

- **1 The lambda expression is extracted into a method.**

You need to add the method `getCaloricLevel` inside the `Dish` class itself for this code to work:

```
public class Dish{
    ...
    public CaloricLevel getCaloricLevel() {
        if (this.getCalories() <= 400) return CaloricLevel.DIET;
        else if (this.getCalories() <= 700) return CaloricLevel.NORMAL;
        else return CaloricLevel.FAT;
    }
}
```

In addition, consider using helper static methods such as `comparing` and `maxBy` whenever possible. These methods were designed for use with method references! Indeed, this code states much more clearly its intent than its counterpart using a lambda expression, as we showed you in [chapter 3](#):

```
inventory.sort(
    (Apple a1, Apple a2) -> a1.getWeight().compareTo(a2.getWeight()));
inventory.sort(comparing(Apple::getWeight));
```

- **1 You need to think about the implementation of comparison.**
- **2 Reads like the problem statement**

Moreover, for many common reduction operations such as *sum*, *maximum*, there are built-in helper methods that can be combined with method references. We showed you, for example, that by using the `Collectors` API, you can find the maximum or sum in a clearer way than by using a combination of a lambda expression and a lower-level `reduce` operation. Instead of writing

```
int totalCalories =
    menu.stream().map(Dish::getCalories)
        .reduce(0, (c1, c2) -> c1 + c2);
```

try using alternative built-in collectors, which state the problem statement more clearly. Here, we use the collector `summingInt` (names go a long way in documenting your code):

Other formats available



✕ `.stream().collect(summingInt(Dish::getCalories));`

Processing to Streams

Ideally, you should try to convert all code that processes a collection with typical data processing patterns with an iterator to use the `Streams` API instead. Why? The `Streams` API expresses more clearly the intent of a

data processing pipeline. In addition, streams can be optimized behind the scenes, making use of short-circuiting and laziness as well as leveraging your multicore architecture, as we explained in [chapter 7](#).

The following imperative code expresses two patterns (filtering and extracting) that are mangled together, forcing the programmer to carefully figure out the whole implementation before figuring out what the code does. In addition, an implementation that executes in parallel would be a lot more difficult to write. See [chapter 7](#) (particularly [section 7.2](#)) to get an idea of the work involved:

```
List<String> dishNames = new ArrayList<>();
for(Dish dish: menu) {
    if(dish.getCalories() > 300){
        dishNames.add(dish.getName());
    }
}
```

The alternative, which uses the Streams API, reads more like the problem statement, and it can be easily parallelized:

```
menu.parallelStream()
    .filter(d -> d.getCalories() > 300)
    .map(Dish::getName)
    .collect(toList());
```

Unfortunately, converting imperative code to the Streams API can be a difficult task, because you need to think about control-flow statements such as `break`, `continue`, and `return` and then infer the right stream operations to use. The good news is that some tools can help you with this task as well. The good news is that some tools (e.g., Lambda-Ficator, <https://ieeexplore.ieee.org/document/6606699>) can help you with this task as well.

9.1.5. Improving code flexibility

We argued in [chapters 2](#) and [3](#) that lambda expressions encourage the style of behavior parameterization. You can represent multiple behaviors with different lambdas that you can then pass around to execute. This style lets you cope with requirement changes (creating multiple ways of filtering with a `Predicate` or comparing with a `Comparator`, for exam-

Other formats available



Audiobook



at a couple of patterns that you can apply immediately from lambda expressions.

First, you can't use lambda expressions without functional interfaces; therefore, you should start introducing them in your code base. But in which situations should you introduce them? In this chapter, we discuss

two common code patterns that can be refactored to leverage lambda expressions: conditional deferred execution and execute around. Also, in the next section, we show you how various object-oriented design patterns—such as the strategy and template-method design patterns—can be rewritten more concisely with lambda expressions.

Conditional deferred execution

It's common to see control-flow statements mangled inside business-logic code. Typical scenarios include security checks and logging. Consider the following code, which uses the built-in Java `Logger` class:

```
if (logger.isLoggable(Log.FINER)) {
    logger.finer("Problem: " + generateDiagnostic());
}
```

What's wrong with it? A couple of things:

- The state of the logger (what level it supports) is exposed in the client code through the method `isLoggable`.
- Why should you have to query the state of the logger object every time before you log a message? It clutters your code.

A better alternative is to use the `log` method, which checks internally to see whether the logger object is set to the right level before logging the message:

```
logger.log(Level.FINER, "Problem: " + generateDiagnostic());
```

This approach is better because your code isn't cluttered with `if` checks, and the state of the logger is no longer exposed. Unfortunately, this code still has an issue: the logging message is always evaluated, even if the logger isn't enabled for the message level passed as an argument.

Lambda expressions can help. What you need is a way to defer the construction of the message so that it can be generated only under a given condition (here, when the logger level is set to `FINER`). It turns out that the Java 8 API designers knew about this problem and introduced an overloaded alternative to `log` that takes a `Supplier` as an argument. This alternative `log` method has the following signature:

Other formats available

 Audiobook 

`logger.log(Level.FINER, () -> "Problem: " + generateDiagnostic());`

Now you can call it as follows:

```
logger.log(Level.FINER, () -> "Problem: " + generateDiagnostic());
```

The `log` method internally executes the lambda passed as an argument only if the logger is of the right level. The internal implementation of the `log` method is along these lines:

```
public void log(Level level, Supplier<String> msgSupplier) {  
    if(logger.isLoggable(level)){  
        log(level, msgSupplier.get());  
    }  
}
```

- **1 Executing the lambda**

What's the takeaway from the story? If you see yourself querying the state of an object (such as the state of the logger) many times in client code, only to call some method on this object with arguments (such as to log a message), consider introducing a new method that calls that method, passed as a lambda or method reference, only after internally checking the state of the object. Your code will be more readable (less cluttered) and better encapsulated, without exposing the state of the object in client code.

Execute around

In [chapter 3](#), we discussed another pattern that you can adopt: execute around. If you find yourself surrounding different code with the same preparation and cleanup phases, you can often pull that code into a lambda. The benefit is that you can reuse the logic dealing with the preparation and cleanup phases, thus reducing code duplication.

Here's the code that you saw in [chapter 3](#). It reuses the same logic to open and close a file but can be parameterized with different lambdas to process the file:

```
String oneLine =  
    processFile((BufferedReader b) -> b.readLine());  
String twoLines =  
    processFile((BufferedReader b) -> b.readLine() + b.readLine());  
public static String processFile(BufferedReaderProcessor p) throws  
    IOException {  
    try(BufferedReader br = new BufferedReader(new  
        FileReader("ModernJavaInAction/chap9/data.txt"))) {  
        return p.process(br);  
    }  
}
```

Other formats available



- **1 Pass a lambda.**
- **2 Pass a different lambda.**

- **3 Execute the `BufferedReaderProcessor` passed as an argument.**
- **4 A functional interface for a lambda, which can throw an `IOException`**

This code was made possible by introducing the functional interface `BufferedReaderProcessor`, which lets you pass different lambdas to work with a `BufferedReader` object.

In this section, you've seen how to apply various recipes to improve the readability and flexibility of your code. In the next section, you see how lambda expressions can remove boilerplate code associated with common object-oriented design patterns.

9.2. Refactoring object-oriented design patterns with lambdas

New language features often make existing code patterns or idioms less popular. The introduction of the `for-each` loop in Java 5, for example, has replaced many uses of explicit iterators because it's less error-prone and more concise. The introduction of the diamond operator `<>` in Java 7 reduced the use of explicit generics at instance creation (and slowly pushed Java programmers to embrace type inference).

A specific class of patterns is called design patterns.^[2] *Design patterns* are reusable blueprints, if you will, for common problems in designing software. They are rather like how construction engineers have a set of reusable solutions to construct bridges for specific scenarios (suspension bridge, arch bridge, and so on). The *visitor design pattern*, for example, is a common solution for separating an algorithm from a structure on which it needs to operate. The *singleton pattern* is a common solution to restrict the instantiation of a class to one object.

²

See Design Patterns: Elements of Reusable Object-Oriented Software, by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides; ISBN 978-0201633610, ISBN 0-201-63361-2

Lambda expressions provide yet another new tool in the programmer's toolbox. They can provide alternative solutions to the problems that the design patterns are tackling, but often with less work and in a simpler way. Many existing object-oriented design patterns can be made redundant or written in a more concise way with lambda expressions.

Other formats available

 Audiobook 

design patterns:

- Template method
- Observer
- Chain of responsibility
- Factory

We show you how lambda expressions can provide an alternative way to solve the problem that each design pattern is intended to solve.

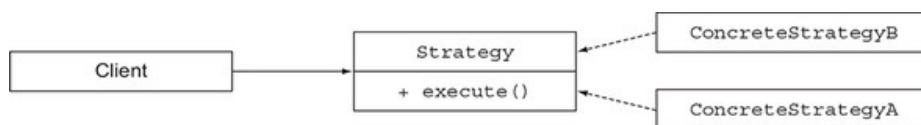
9.2.1. Strategy

The strategy pattern is a common solution for representing a family of algorithms and letting you choose among them at runtime. You saw this pattern briefly in [chapter 2](#) when we showed you how to filter an inventory with different predicates (such as heavy apples or green apples). You can apply this pattern to a multitude of scenarios, such as validating an input with different criteria, using different ways of parsing, or formatting an input.

The strategy pattern consists of three parts, as illustrated in [figure 9.1](#):

- An interface to represent some algorithm (the interface Strategy)
- One or more concrete implementations of that interface to represent multiple algorithms (the concrete classes ConcreteStrategyA, ConcreteStrategyB)
- One or more clients that use the strategy objects

Figure 9.1. The strategy design pattern



Suppose that you'd like to validate whether a text input is properly formatted for different criteria (consists of only lowercase letters or is numeric, for example). You start by defining an interface to validate the text (represented as a String):

```
public interface ValidationStrategy {
    boolean execute(String s);
}
```

Second, you define one or more implementation(s) of that interface:

```
public class IsAllLowerCase implements ValidationStrategy {
    public boolean execute(String s){
        return s.matches("[a-z]+");
    }
}
```

Other formats available



Audiobook



```
implements ValidationStrategy {
    te(String s){
        ("\\d+");
    }
}
```

Then you can use these different validation strategies in your program:

```
public class Validator {
    private final ValidationStrategy strategy;
    public Validator(ValidationStrategy v) {
        this.strategy = v;
    }
    public boolean validate(String s) {
        return strategy.execute(s);
    }
}

Validator numericValidator = new Validator(new IsNumeric());
boolean b1 = numericValidator.validate("aaaa");           1
Validator lowerCaseValidator = new Validator(new IsAllLowerCase ());
boolean b2 = lowerCaseValidator.validate("bbbb");         2
```

- **1 Returns false**
- **2 Returns true**

Using lambda expressions

By now, you should recognize that `ValidationStrategy` is a functional interface. In addition, it has the same function descriptor as `Predicate<String>`. As a result, instead of declaring new classes to implement different strategies, you can pass more concise lambda expressions directly:

```
Validator numericValidator =
    new Validator((String s) -> s.matches("[a-z]+"));      1
boolean b1 = numericValidator.validate("aaaa");
Validator lowerCaseValidator =
    new Validator((String s) -> s.matches("\\d+"));        1
boolean b2 = lowerCaseValidator.validate("bbbb");
```

- **1 Passing a lambda directly**

As you can see, lambda expressions remove the boilerplate code that's inherent to the strategy design pattern. If you think about it, lambda expressions encapsulate a piece of code (or strategy), which is what the strategy design pattern was created for, so we recommend that you use lambda expressions instead for similar problems.

Other formats available



... is a common solution when you
... an algorithm and have the additional
flexibility to change certain parts of it. Okay, this pattern sounds a bit abstract. In other words, the template method pattern is useful when you find yourself saying “I’d love to use this algorithm, but I need to change a few lines so it does what I want.”

Here's an example of how this pattern works. Suppose that you need to write a simple online banking application. Users typically enter a customer ID; the application fetches the customer's details from the bank's database and does something to make the customer happy. Different online banking applications for different banking branches may have different ways of making a customer happy (such as adding a bonus to his account or sending him less paperwork). You can write the following abstract class to represent the online banking application:

```
abstract class OnlineBanking {
    public void processCustomer(int id){
        Customer c = Database.getCustomerWithId(id);
        makeCustomerHappy(c);
    }
    abstract void makeCustomerHappy(Customer c);
}
```

The `processCustomer` method provides a sketch for the online banking algorithm: Fetch the customer given its ID and make the customer happy. Now different branches can provide different implementations of the `makeCustomerHappy` method by subclassing the `OnlineBanking` class.

Using lambda expressions

You can tackle the same problem (creating an outline of an algorithm and letting implementers plug in some parts) by using your favorite lambdas. The components of the algorithms you want to plug in can be represented by lambda expressions or method references.

Here, we introduce a second argument to the `processCustomer` method of type `Consumer<Customer>` because it matches the signature of the method `makeCustomerHappy` defined earlier:

```
public void processCustomer(int id, Consumer<Customer> makeCustomerHappy)
    Customer c = Database.getCustomerWithId(id);
    makeCustomerHappy.accept(c);
}
```

Now you can plug in different behaviors directly without subclassing the `OnlineBanking` class by passing lambda expressions:

Other formats available



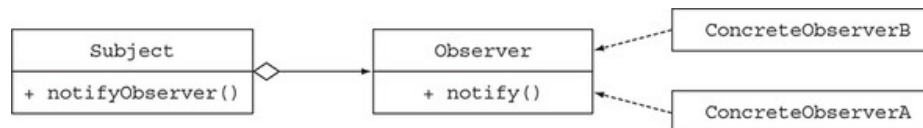
✕ `).processCustomer(1337, (Customer c) -> "Hello " + c.getName());`

This example shows how lambda expressions can help you remove the boilerplate inherent to design patterns.

9.2.3. Observer

The observer design pattern is a common solution when an object (called the *subject*) needs to automatically notify a list of other objects (called *observers*) when some event happens (such as a state change). You typically come across this pattern when working with GUI applications. You register a set of observers on a GUI component such as a button. If the button is clicked, the observers are notified and can execute a specific action. But the observer pattern isn't limited to GUIs. The observer design pattern is also suitable in a situation in which several traders (observers) want to react to the change of price of a stock (subject). [Figure 9.2](#) illustrates the UML diagram of the observer pattern.

Figure 9.2. The observer design pattern



Now write some code to see how useful the observer pattern is in practice. You'll design and implement a customized notification system for an application such as Twitter. The concept is simple: several newspaper agencies (*The New York Times*, *The Guardian*, and *Le Monde*) are subscribed to a feed of news tweets and may want to receive a notification if a tweet contains a particular keyword.

First, you need an `Observer` interface that groups the observers. It has one method, called `notify`, that will be called by the subject (`Feed`) when a new tweet is available:

```
interface Observer {
    void notify(String tweet);
}
```

Now you can declare different observers (here, the three newspapers) that produce a different action for each different keyword contained in a tweet:

```
class NYTimes implements Observer {
    public void notify(String tweet) {
        if(tweet != null && tweet.contains("money")){
            System.out.println("Breaking news in NY! " + tweet);
        }
    }
}
```

Other formats available



Audiobook



```

class LondonObserver implements Observer {
    public void notify(String tweet) {
        if(tweet != null && tweet.contains("queen")){
            System.out.println("Yet more news from London... " + tweet);
        }
    }
}
}
}

```

```

class LeMonde implements Observer {
    public void notify(String tweet) {
        if(tweet != null && tweet.contains("wine")){
            System.out.println("Today cheese, wine and news! " + tweet);
        }
    }
}

```

You're still missing the crucial part: the subject. Define an interface for the subject:

```

interface Subject {
    void registerObserver(Observer o);
    void notifyObservers(String tweet);
}

```

The subject can register a new observer using the `registerObserver` method and notify his observers of a tweet with the `notifyObservers` method. Now implement the `Feed` class:

```

class Feed implements Subject {
    private final List<Observer> observers = new ArrayList<>();
    public void registerObserver(Observer o) {
        this.observers.add(o);
    }
    public void notifyObservers(String tweet) {
        observers.forEach(o -> o.notify(tweet));
    }
}

```

This implementation is straightforward: the feed keeps an internal list of observers that it can notify when a tweet arrives. You can create a demo application to wire up the subject and observers:

```

Feed f = new Feed();
f.registerObserver(new NYTimes());
f.registerObserver(new Guardian());
f.registerObserver(new LeMonde());
f.notifyObservers("The queen said her favourite book is Modern Java in Ac

```

Unsurprisingly, *The Guardian* picks up this tweet.

Other formats available

 Audiobook 

...the lambda expressions with the observer design pattern. Notice that the various classes that implement the `Observer` interface all provide implementation for a single method: `notify`. They're wrapping a piece of behavior to execute when a tweet arrives. Lambda expressions are designed specifically to remove that boilerplate.

erplate. Instead of instantiating three observer objects explicitly, you can pass a lambda expression directly to represent the behavior to execute:

```
f.registerObserver((String tweet) -> {
    if(tweet != null && tweet.contains("money")){
        System.out.println("Breaking news in NY! " + tweet);
    }
});
f.registerObserver((String tweet) -> {
    if(tweet != null && tweet.contains("queen")){
        System.out.println("Yet more news from London... " + tweet);
    }
});
```

Should you use lambda expressions all the time? The answer is no. In the example we described, lambda expressions work great because the behavior to execute is simple, so they're helpful for removing boilerplate code. But the observers may be more complex; they could have state, define several methods, and the like. In those situations, you should stick with classes.

9.2.4. Chain of responsibility

The chain of responsibility pattern is a common solution to create a chain of processing objects (such as a chain of operations). One processing object may do some work and pass the result to another object, which also does some work and passes it on to yet another processing object, and so on.

Generally, this pattern is implemented by defining an abstract class representing a processing object that defines a field to keep track of a successor. When it finishes its work, the processing object hands over its work to its successor. The code looks like this:

```
public abstract class ProcessingObject<T> {
    protected ProcessingObject<T> successor;
    public void setSuccessor(ProcessingObject<T> successor){
        this.successor = successor;
    }
    public T handle(T input) {
        T r = handleWork(input);
        if(successor != null){
            return successor.handle(r);
        }
        return r;
    }
    abstract T handleWork(T input);
}
```

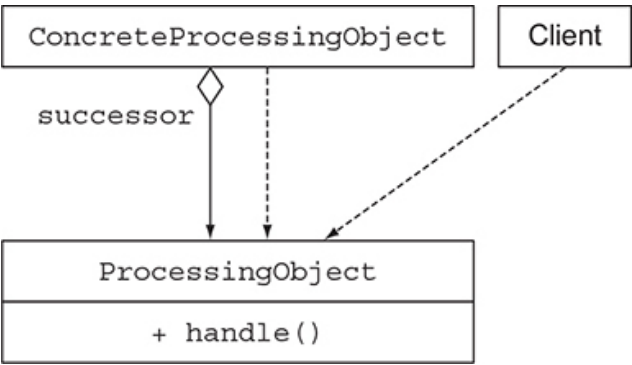
Other formats available

 Audiobook 

handleWork(T input);

Figure 9.3 illustrates the chain of responsibility pattern in UML.

Figure 9.3. The chain of responsibility design pattern



Here, you may recognize the template method design pattern, which we discussed in [section 9.2.2](#). The `handle` method provides an outline for dealing with a piece of work. You can create different kinds of processing objects by subclassing the `ProcessingObject` class and by providing an implementation for the `handleWork` method.

Here’s an example of how to use this pattern. You can create two processing objects doing some text processing:

```
public class HeaderTextProcessing extends ProcessingObject<String> {
    public String handleWork(String text) {
        return "From Raoul, Mario and Alan: " + text;
    }
}

public class SpellCheckerProcessing extends ProcessingObject<String> {
    public String handleWork(String text) {
        return text.replaceAll("labda", "lambda");
    }
}
```

- **1 Oops—we forgot the ‘m’ in “lambda”!**

Now you can connect two processing objects to construct a chain of operations:

```
ProcessingObject<String> p1 = new HeaderTextProcessing();
ProcessingObject<String> p2 = new SpellCheckerProcessing();
p1.setSuccessor(p2);
String result = p1.handle("Aren't labdas really sexy?!");
System.out.println(result);
```

Other formats available

Audiobook

ConcreteProcessingObject
Client
ProcessingObject
+ handle()
successor
1
2
Objects
and Alan: Aren’t lambdas really

Wait a minute—this pattern looks like chaining (that is, composing) functions. We discussed composing lambda expressions in [chapter 3](#). You can represent the processing objects as an instance of `Function<String, String>`, or (more precisely) a `UnaryOperator<String>`. To chain them, compose these functions by using the `andThen` method:

```
UnaryOperator<String> headerProcessing =  
    (String text) -> "From Raoul, Mario and Alan: " + text;      1  
UnaryOperator<String> spellCheckerProcessing =  
    (String text) -> text.replaceAll("labda", "lambda");          2  
Function<String, String> pipeline =  
    headerProcessing.andThen(spellCheckerProcessing);              3  
String result = pipeline.apply("Aren't labdas really sexy?!");
```

- **1 The first processing object**
- **2 The second processing object**
- **3 Compose the two functions, resulting in a chain of operations.**

9.2.5. Factory

The factory design pattern lets you create objects without exposing the instantiation logic to the client. Suppose that you're working for a bank that needs a way of creating different financial products: loans, bonds, stocks, and so on.

Typically, you'd create a `Factory` class with a method that's responsible for the creation of different objects, as shown here:

```
public class ProductFactory {  
    public static Product createProduct(String name) {  
        switch(name){  
            case "loan": return new Loan();  
            case "stock": return new Stock();  
            case "bond": return new Bond();  
            default: throw new RuntimeException("No such product " + name  
        }  
    }  
}
```

Here, `Loan`, `Stock`, and `Bond` are subtypes of `Product`. The `createProduct` method could have additional logic to configure each created product. But the benefit is that you can create these objects without exposing

Other formats available



tion to the client, which makes the creation of the client, as follows:

```
Product p = ProductFactory.createProduct("loan");
```

Using lambda expressions

You saw in [chapter 3](#) that you can refer to constructors the way that you refer to methods: by using method references. Here's how to refer to the `Loan` constructor:

```
Supplier<Product> loanSupplier = Loan::new;
Loan loan = loanSupplier.get();
```

Using this technique, you could rewrite the preceding code by creating a `Map` that maps a product name to its constructor:

```
final static Map<String, Supplier<Product>> map = new HashMap<>();
static {
    map.put("loan", Loan::new);
    map.put("stock", Stock::new);
    map.put("bond", Bond::new);
}
```

You can use this `Map` to instantiate different products, as you did with the factory design pattern:

```
public static Product createProduct(String name){
    Supplier<Product> p = map.get(name);
    if(p != null) return p.get();
    throw new IllegalArgumentException("No such product " + name);
}
```

This technique is a neat way to use this Java 8 feature to achieve the same intent as the factory pattern. But this technique doesn't scale well if the factory method `create-Product` needs to take multiple arguments to pass to the product constructors. You'd have to provide a functional interface other than a simple `Supplier`.

Suppose that you want to refer to constructors for products that take three arguments (two `Integers` and a `String`); you need to create a special functional interface `TriFunction` to support such constructors. As a result, the signature of the `Map` becomes more complex:

```
public interface TriFunction<T, U, V, R> {
    R apply(T t, U u, V v);
}
```

✕ `Integer, Integer, String, Product>> map`

Other formats available

 Audiobook 

You've seen how to write and refactor code by using lambda expressions. In the next section, you see how to ensure that your new code is correct.

9.3. Testing lambdas

You've sprinkled your code with lambda expressions, and it looks nice and concise. But in most developer jobs, you're paid not for writing nice code, but for writing code that's correct.

Generally, good software engineering practice involves using unit testing to ensure that your program behaves as intended. You write test cases, which assert that small individual parts of your source code are producing the expected results. Consider a simple `Point` class for a graphical application:

```
public class Point {
    private final int x;
    private final int y;
    private Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public int getX() { return x; }
    public int getY() { return y; }
    public Point moveRightBy(int x) {
        return new Point(this.x + x, this.y);
    }
}
```

The following unit test checks whether the method `moveRightBy` behaves as expected:

```
@Test
public void testMoveRightBy() throws Exception {
    Point p1 = new Point(5, 5);
    Point p2 = p1.moveRightBy(10);
    assertEquals(15, p2.getX());
    assertEquals(5, p2.getY());
}
```

9.3.1. Testing the behavior of a visible lambda

This code works nicely because the `moveRightBy` method is public and, therefore, can be tested inside the test case. But lambdas don't have names (they're anonymous functions, after all), and testing them in your code is tricky because you can't refer to them by name.

Other formats available



lambda via a field so that you can reuse encapsulated in that lambda. What can as you do when calling methods. compareByXAndThenY in the Point

class that gives you access to a `Comparator` object generated from method references:

```
public class Point {
    public final static Comparator<Point> compareByXAndThenY =
        comparing(Point::getX).thenComparing(Point::getY);
    ...
}
```

Remember that lambda expressions generate an instance of a functional interface. As a result, you can test the behavior of that instance. Here, you can call the `compare` method on the `Comparator` object `compareByXAndThenY` with different arguments to test whether its behavior is as intended:

```
@Test
public void testComparingTwoPoints() throws Exception {
    Point p1 = new Point(10, 15);
    Point p2 = new Point(10, 20);
    int result = Point.compareByXAndThenY.compare(p1, p2);
    assertTrue(result < 0);
}
```

9.3.2. Focusing on the behavior of the method using a lambda

But the purpose of lambdas is to encapsulate a one-off piece of behavior to be used by another method. In that case, you shouldn't make lambda expressions available publicly; they're only implementation details. Instead, we argue that you should test the behavior of the method that uses a lambda expression. Consider the `moveAllPointsRightBy` method shown here:

```
public static List<Point> moveAllPointsRightBy(List<Point> points, int x)
    return points.stream()
        .map(p -> new Point(p.getX() + x, p.getY()))
        .collect(toList());
}
```

There's no point (pun intended) in testing the lambda `p -> new Point(p.getX() + x, p.getY())`; it's only an implementation detail for the `moveAllPointsRightBy` method. Instead, you should focus on testing the behavior of the `moveAllPointsRightBy` method:

Other formats available

 Audiobook 

```
public void testMoveAllPointsRightBy() throws Exception {
    List<Point> points =
        Arrays.asList(new Point(5, 5), new Point(10, 5));
    List<Point> expectedPoints =
        Arrays.asList(new Point(15, 5), new Point(20, 5));
    List<Point> newPoints = Point.moveAllPointsRightBy(points, 10);
}
```

```

        assertEquals(expectedPoints, newPoints);
    }
}

```

Note that in the unit test, it's important for the `Point` class to implement the `equals` method appropriately; otherwise, it relies on the default implementation from `Object`.

9.3.3. Pulling complex lambdas into separate methods

Perhaps you come across a really complicated lambda expression that contains a lot of logic (such as a technical pricing algorithm with corner cases). What do you do, because you can't refer to the lambda expression inside your test? One strategy is to convert the lambda expression to a method reference (which involves declaring a new regular method), as we explained in [section 9.1.3](#). Then you can test the behavior of the new method as you would that of any regular method.

9.3.4. Testing high-order functions

Methods that take a function as an argument or return another function (so-called higher-order functions, explained in [chapter 19](#)) are a little harder to deal with. One thing you can do if a method takes a lambda as an argument is test its behavior with different lambdas. You can test the `filter` method that you created in [chapter 2](#) with different predicates:

```

@Test
public void testFilter() throws Exception {
    List<Integer> numbers = Arrays.asList(1, 2, 3, 4);
    List<Integer> even = filter(numbers, i -> i % 2 == 0);
    List<Integer> smallerThanThree = filter(numbers, i -> i < 3);
    assertEquals(Arrays.asList(2, 4), even);
    assertEquals(Arrays.asList(1, 2), smallerThanThree);
}

```

What if the method that needs to be tested returns another function? You can test the behavior of that function by treating it as an instance of a functional interface, as we showed you earlier with a `Comparator`.

Unfortunately, not everything works the first time, and your tests may report some errors related to your use of lambda expressions. So, in the next section we turn to debugging.

Other formats available

 Audiobook 

in old-school weapons for debugging

problematic code:

- Examining the stack trace
- Logging

Lambda expressions and streams can bring new challenges to your typical debugging routine. We explore both in this section.

9.4.1. Examining the stack trace

When your program has stopped (because an exception was thrown, for example), the first thing you need to know is where the program stopped and how it got there. Stack frames are useful for this purpose. Each time your program performs a method call, information about the call is generated, including the location of the call in your program, the arguments of the call, and the local variables of the method being called. This information is stored on a stack frame.

When your program fails, you get a *stack trace*, which is a summary of how your program got to that failure, stack frame by stack frame. In other words, you get a valuable list of method calls up to when the failure appeared. This list helps you understand how the problem occurred.

Using lambda expressions

Unfortunately, due to the fact that lambda expressions don't have names, stack traces can be slightly puzzling. Consider the following simple code, which is made to fail on purpose:

```
import java.util.*;
public class Debugging{
    public static void main(String[] args) {
        List<Point> points = Arrays.asList(new Point(12, 2), null);
        points.stream().map(p -> p.getX()).forEach(System.out::println);
    }
}
```

Running this code produces a stack trace along the lines of the following (depending on your javac version; you may not have the same stack trace):

```
Exception in thread "main" java.lang.NullPointerException
    at Debugging.lambda$main$0(Debugging.java:6)
    at Debugging$$Lambda$5/284720968.apply(Unknown Source)
    at java.util.stream.ReferencePipeline$3$1.accept(ReferencePipeline
        .java:193)
    at java.util.Spliterators$ArraySpliterator.forEachRemaining(Spliterat
        java:849)
```

Other formats available

 Audiobook 

Yuck! What's going on? The program fails, of course, because the second element of the list of points is `null`. You try to process a `null` reference. Because the error occurs in a stream pipeline, the whole sequence of

method calls that make a stream pipeline work is exposed to you. But notice that the stack trace produces the following cryptic lines:

```
at Debugging.lambda$main$0(Debugging.java:6)
    at Debugging$$Lambda$5/284720968.apply(Unknown Source)
```

These lines mean that the error occurred inside a lambda expression. Unfortunately, because lambda expressions don't have names, the compiler has to make up a name to refer to them. In this case, the name is `lambda$main$0`, which isn't intuitive and can be problematic if you have large classes containing several lambda expressions.

Even if you use method references, it's still possible that the stack won't show you the name of the method you used. Changing the preceding `lambda p -> p.getX()` to the method reference `Point::getX` also results in a problematic stack trace:

```
points.stream().map(Point::getX).forEach(System.out::println);
Exception in thread "main" java.lang.NullPointerException
    at Debugging$$Lambda$5/284720968.apply(Unknown Source)
    at java.util.stream.ReferencePipeline$3$1.accept(ReferencePipeline
        .java:193)
    ...
```

- **1 What does this line mean?**

Note that if a method reference refers to a method declared in the same class where it's used, it appears in the stack trace. In the following example:

```
import java.util.*;
public class Debugging{
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3);
        numbers.stream().map(Debugging::divideByZero).forEach(System
            .out::println);
    }
    public static int divideByZero(int n){
        return n / 0;
    }
}
```

Other formats available

 Audiobook 



orted correctly in the stack trace:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Debugging.divideByZero(Debugging.java:10)
    at Debugging$$Lambda$1/999966131.apply(Unknown Source)
    at java.util.stream.ReferencePipeline$3$1.accept(ReferencePipeline
```

```
.java:193)
...
```

- **1 divideByZero appears in the stack trace.**

In general, keep in mind that stack traces involving lambda expressions may be more difficult to understand. This area is one in which the compiler can be improved in a future version of Java.

9.4.2. Logging information

Suppose that you're trying to debug a pipeline of operations in a stream. What can you do? You could use `forEach` to print or log the result of a stream as follows:

```
List<Integer> numbers = Arrays.asList(2, 3, 4, 5);
numbers.stream()
    .map(x -> x + 17)
    .filter(x -> x % 2 == 0)
    .limit(3)
    .forEach(System.out::println);
```

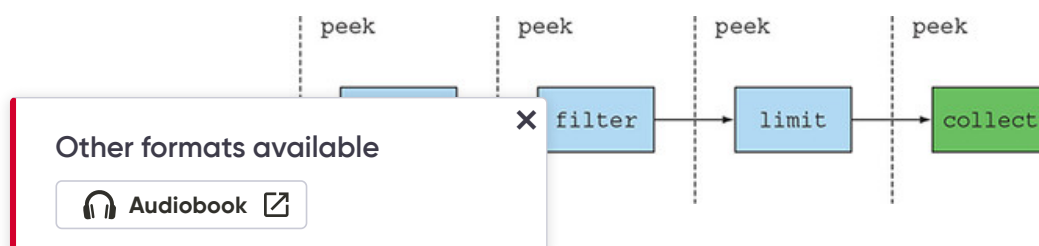
This code produces the following output:

```
20
22
```

Unfortunately, after you call `forEach`, the whole stream is consumed. It would be useful to understand what each operation (`map`, `filter`, `limit`) produces in the pipeline of a stream.

The stream operation `peek` can help. The purpose of `peek` is to execute an action on each element of a stream as it's consumed. It doesn't consume the whole stream the way `forEach` does, however; it forwards the element on which it performed an action to the next operation in the pipeline. [Figure 9.4](#) illustrates the `peek` operation.

Figure 9.4. Examining values flowing in a stream pipeline with `peek`



In the following code, you use `peek` to print the intermediate values before and after each operation in the stream pipeline:


```
List<Integer> result =
    numbers.stream()
        .peek(x -> System.out.println("from stream: " + x))      1
        .map(x -> x + 17)
        .peek(x -> System.out.println("after map: " + x))        2
        .filter(x -> x % 2 == 0)
        .peek(x -> System.out.println("after filter: " + x))     3
        .limit(3)
        .peek(x -> System.out.println("after limit: " + x))      4
        .collect(toList());
```

- **1 Print the current element consumed from the source**
- **2 Print the result of the map operation.**
- **3 Print the number selected after the filter operation.**
- **4 Print the number selected after the limit operation.**

This code produces useful output at each step of the pipeline:

```
from stream: 2
after map: 19
from stream: 3
after map: 20
after filter: 20
after limit: 20
from stream: 4
after map: 21
from stream: 5
after map: 22
after filter: 22
after limit: 22
```

Summary

- Lambda expressions can make your code more readable and flexible.
- Consider converting anonymous classes to lambda expressions, but be wary of subtle semantic differences such as the meaning of the keyword `this` and shadowing of variables.
- Method references can make your code more readable compared with lambda expressions.
- Consider converting iterative collection processing to use the Streams API.
- Lambda expressions can remove boilerplate code associated with sev-

Other formats available



- terns, such as strategy, template responsibility, and factory.
- nit-tested, but in general, you should fo-
- the methods in which the lambda ex-
- pressions appear.
- Consider extracting complex lambda expressions into regular methods.
- Lambda expressions can make stack traces less readable.

- The peek method of a stream is useful for logging intermediate values as they flow past certain points of a stream pipeline.

Other formats available



Audiobook

